

```
In [3]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

from sklearn.metrics import confusion_matrix
from sklearn.metrics import recall_score
from sklearn.metrics import precision_score
from sklearn.metrics import accuracy_score
```

```
In [4]: df = pd.read_csv('Social_Network_Ads.csv')
```

```
In [5]: df.head()
```

```
Out[5]:
```

	Age	EstimatedSalary	Purchased
0	19	19000	0
1	35	20000	0
2	26	43000	0
3	27	57000	0
4	19	76000	0

```
In [6]: df.describe
```

```
Out[6]: <bound method NDFrame.describe of
0    19    19000    0
1    35    20000    0
2    26    43000    0
3    27    57000    0
4    19    76000    0
..    ...    ...    ...
395  46    41000    1
396  51    23000    1
397  50    20000    1
398  36    33000    0
399  49    36000    1

[400 rows x 3 columns]>
```

```
In [7]: X=df[['Age','EstimatedSalary']]
```

```
In [9]: y=df['Purchased']
```

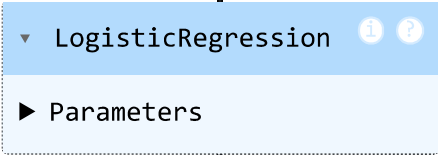
```
In [10]: X_train,X_test,y_train,y_test=train_test_split(
    X,y,test_size=0.25,random_state=42
)
```

```
In [13]: sc = StandardScaler()

X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

```
In [14]: model = LogisticRegression()
```

```
In [15]: model.fit(X_train,y_train)
```

```
Out[15]: 
  ▾ LogisticRegression ⓘ ?
    ▶ Parameters
```

```
In [16]: y_pred = model.predict(X_test)
```

```
In [18]: cm = confusion_matrix(y_test,y_pred)
print(cm)
```

```
[[61  2]
 [12 25]]
```

```
In [19]: TN = cm[0][0]
FP = cm[0][1]
FN = cm[1][0]
TP = cm[1][1]

print("TN : ",TN)
print('FP : ',FP)
print('FN : ',FN)
print('TP : ',TP)
```

```
TN :  61
FP :   2
FN :  12
TP :  25
```

```
In [21]: accuracy = accuracy_score(y_test,y_pred)
print(accuracy)
```

```
0.86
```

```
In [22]: error_rate = 1-accuracy
print(error_rate)
```

```
0.14
```

```
In [23]: precision = precision_score(y_test,y_pred)
print(precision)
```

```
0.9259259259259259
```

```
In [24]: recall = recall_score(y_test,y_pred)
print(recall)
```

```
0.6756756756756757
```

DSBDA Practical 5 — Data Analytics II

Logistic Regression on Social_Network_Ads Dataset

Dataset Source: [Social Network Ads Dataset Kaggle](#)

Aim

To implement Logistic Regression for classification and calculate:

- Confusion Matrix
 - TP, FP, TN, FN
 - Accuracy
 - Error Rate
 - Precision
 - Recall
-

What is Logistic Regression?

Logistic Regression is a supervised machine learning algorithm used for:

- Classification problems
- Predicting categories/classes

Example:

- Spam detection
 - Disease prediction
 - Purchase prediction
-

Why Called Regression?

Although used for classification, it predicts probability using regression concepts.

Logistic Function (Sigmoid Function)

Formula:

$$P(y=1)=\frac{1}{1+e^{-z}}$$

Where:

- $P(y=1)$ = probability
 - e = exponential
 - z = linear combination of inputs
-

Output Range

Sigmoid converts output between:

0 and 1

About Dataset

Social_Network_Ads dataset contains:

- Age
- Estimated Salary
- Purchased (Target)

Goal: Predict whether customer purchases product.

Step 1: Import Libraries

Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import confusion_matrix
```

```
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
```

Explanation

Library	Purpose
pandas	Data handling
numpy	Numerical operations
matplotlib	Visualization
train_test_split	Train-test division
StandardScaler	Feature scaling
LogisticRegression	Classification model
metrics	Evaluation

Viva Questions

What is Scikit-learn?

Python ML library.

Why scaling required?

To bring features to same scale.

Step 2: Load Dataset

Code

```
df = pd.read_csv("Social_Network_Ads.csv")
```

Display Dataset

Code

```
df.head()
```

Step 3: Select Features and Target

Features

Code

```
X = df[['Age', 'EstimatedSalary']]
```

Target Variable

Code

```
y = df['Purchased']
```

Explanation

Variable	Meaning
X	Independent variables
y	Dependent variable

Step 4: Split Dataset

Code

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.25, random_state=0  
)
```

Explanation

```
test_size=0.25
```

25% testing data.

```
random_state=0
```

Ensures same random split every run.

Step 5: Feature Scaling

Code

```
sc = StandardScaler()

X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

Explanation

Scaling standardizes data.

Formula:

genui☆
$$z = \frac{x - \mu}{\sigma}$$

Where:

- μ = mean
 - σ = standard deviation
-

Why Scaling Needed?

Because:

- Age and Salary have different ranges
 - Logistic Regression performs better on scaled data
-

Viva Questions

What is feature scaling?

Transforming features to similar scale.

Difference between `fit_transform()` and `transform()`?

Method	Purpose
<code>fit_transform</code>	Learn + transform
<code>transform</code>	Only transform

Step 6: Create Logistic Regression Model

Code

```
model = LogisticRegression()
```

Explanation

Creates Logistic Regression classifier.

Step 7: Train Model

Code

```
model.fit(X_train, y_train)
```

Explanation

Model learns relationship between:

- Features
 - Purchased status
-

Step 8: Predict Test Results

Code

```
y_pred = model.predict(X_test)
```

Explanation

Predicts:

0 or 1

Where:

- 0 = Not Purchased
 - 1 = Purchased
-

Step 9: Create Confusion Matrix

Code

```
cm = confusion_matrix(y_test, y_pred)

print(cm)
```

Example Output

```
[[ 65  3]
 [ 8 24]]
```

Confusion Matrix Theory

Actual/Predicted	Positive	Negative
Positive	TP	FN
Negative	FP	TN

Meaning

Term	Meaning
TP	Correct positive prediction
TN	Correct negative prediction
FP	Wrong positive prediction
FN	Wrong negative prediction

Extract TP, FP, TN, FN

Code

```
TN = cm[0][0]
FP = cm[0][1]
FN = cm[1][0]
TP = cm[1][1]

print("TP =", TP)
print("TN =", TN)
print("FP =", FP)
print("FN =", FN)
```

Step 10: Accuracy

Code

```
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy =", accuracy)
```

Accuracy Formula

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

Explanation

Accuracy measures:

- Correct predictions percentage
-

Step 11: Error Rate

Code

```
error_rate = 1 - accuracy  
  
print("Error Rate =", error_rate)
```

Error Rate Formula

Error\ Rate = $\frac{FP+FN}{Total}$

Step 12: Precision

Code

```
precision = precision_score(y_test, y_pred)  
  
print("Precision =", precision)
```

Precision Formula

Precision = $\frac{TP}{TP+FP}$

Explanation

Precision measures:

- Correctness of positive predictions
-

Step 13: Recall

Code

```
recall = recall_score(y_test, y_pred)

print("Recall =", recall)
```

Recall Formula

$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$

Explanation

Recall measures:

- Ability to find actual positives
-

Important Theory

What is Classification?

Predicting categories/classes.

Examples:

- Yes/No
 - Spam/Not Spam
-

Difference Between Regression and Classification

Regression	Classification
Continuous output	Categorical output
Example: price	Example: purchased/not

What is Sigmoid Function?

Converts values into probability between:

0 and 1

Threshold in Logistic Regression

Usually:

0.5

If probability > 0.5:

Class = 1

Else:

Class = 0

Why Confusion Matrix Important?

Shows detailed model performance.

Important Viva Questions

Basic Questions

What is Logistic Regression?

Classification algorithm.

What type of output does it produce?

Categorical output.

Why feature scaling used?

To normalize feature ranges.

What is confusion matrix?

Table showing prediction performance.

Intermediate Questions

Difference between TP and TN?

TP	TN
Correct positive	Correct negative

Difference between Precision and Recall?

Precision	Recall
Prediction correctness	Finding positives

What is false positive?

Incorrectly predicted positive.

What is false negative?

Incorrectly predicted negative.

Advanced Questions

Why sigmoid function used?

Converts output into probability.

Which metric important in disease detection?

Recall.

Because missing positive case is dangerous.

Which metric important in spam detection?

Precision.

Because false spam detection is problematic.

External May Ask Modifications

Find F1 Score

```
from sklearn.metrics import f1_score  
  
print(f1_score(y_test, y_pred))
```

F1 Score Formula

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Predict Single Value

```
model.predict(sc.transform([[35, 50000]]))
```

Display Probability

```
model.predict_proba(X_test)
```

Plot Confusion Matrix

```
import seaborn as sns  
  
sns.heatmap(cm, annot=True)  
plt.show()
```

Common Exam Errors

Error	Reason
Shape mismatch	Wrong input dimensions
Feature scaling missing	Poor accuracy
Wrong indexing	Confusion matrix extraction mistake

2-Minute Viva Explanation

"In this practical, we implemented Logistic Regression on the Social Network Ads dataset to predict whether a customer purchases a product. We imported required libraries, loaded the dataset, selected features and target variable, and split the data into training and testing sets. We applied feature scaling using StandardScaler and trained the Logistic Regression model using fit(). Then we predicted test outputs and evaluated performance using confusion matrix, accuracy, precision, recall, TP, TN, FP, and FN. Logistic Regression is widely used for binary classification problems."

Excellent viva-type question. Your confusion comes from understanding how the confusion matrix is arranged.

Suppose Confusion Matrix Output is

```
[[65  3]
 [ 8 24]]
```

This matrix is arranged as:

Actual \ Predicted	0 (Negative)	1 (Positive)
0 (Negative)	TN	FP
1 (Positive)	FN	TP

So:

```
[[TN  FP]
 [FN  TP]]
```

Therefore:

```
TN = cm[0][0] = 65
FP = cm[0][1] = 3
FN = cm[1][0] = 8
TP = cm[1][1] = 24
```

Why TP is `[1][1]`?

Because:

- Row index = Actual class
- Column index = Predicted class

And:

`1` = Positive **class**

`0` = Negative **class**

So:

`[1][1]`

means:

- Actual = Positive
- Predicted = Positive

which is True Positive.

Visual Understanding

Confusion Matrix:

		Predicted	
		<code>0</code>	<code>1</code>
Actual	<code>0</code>	TN	FP
	<code>1</code>	FN	TP

Why `[0][0]` is TN and not TP?

Because:

`0` = Negative **class**

So:

`[0][0]`

means:

- Actual Negative
- Predicted Negative

which is True Negative.

Easy Memory Trick

Rows = Actual

Columns = Predicted

And matrix always follows:

```
[[TN FP]
 [FN TP]]
```

Important Viva Question

Q. How is confusion matrix arranged in sklearn?

Answer:

```
[[TN FP]
 [FN TP]]
```

Rows represent actual classes and columns represent predicted classes.

Another Viva Question

Q. Why TP is bottom-right element?

Because:

- Positive class = 1
- Bottom row = actual positive
- Right column = predicted positive

Hence:

```
cm[1][1]
```

is TP.

Quick Example

Suppose:

Actual	Predicted
1	1

This means:

- Actual Positive
- Predicted Positive

Therefore:

TP

stored at:

`cm[1][1]`

Very Common External Question

Q. What does diagonal of confusion matrix represent?

Correct predictions:

- TN
- TP

Off-diagonal:

- Wrong predictions
 - FP and FN
-

Important Point

In binary classification:

`0` → Negative

`1` → Positive

This convention causes TP to be:

`cm[1][1]`

not:

`cm[0][0]`